1

Reproducibility Using VisTrails

Juliana Freire

Polytechnic Institute of New York University

David Koop

Polytechnic Institute of New York University

Fernando Chirigati

Polytechnic Institute of New York University

Cláudio T. Silva

Polytechnic Institute of New York University

CONTENTS

1.1 Introduction

Science has long placed an emphasis on revisiting and reusing past results: reproducibility is a core component of the scientific process. Testing and extending published results are *standard* activities that lead to practical progress: science moves forward using past work and allowing scientists to "stand on the shoulders of giants". In natural science, long tradition requires experiments to be described in enough detail so that they can be reproduced by other researchers. This standard, however, has not been widely applied for computational experiments. Researchers often have to rely on tables, plots and figure captions included in papers. Consequently, it is difficult to verify and reproduce many published results [43], and this has led to a credibility crisis in computational science [17].

Scientific communities in different domains have started to act in an attempt to address this problem. Prestigious conferences such as SIGMOD [58] and VLDB [71], journals such as PNAS [52], Biostatistics [7], the IEEE Transactions on Signal Processing [65], Nature, Science, to name a few, have been encouraging—and sometimes requiring—that published results be accompanied by the necessary data and code needed to reproduce them. However, it can be difficult and time-consuming for authors to make their experiment reproducible and for reviewers to verify the results. Authors need to encapsulate the whole experiment (data, parameter settings, source code and environment) to guarantee that the same results are generated. Even when an experiment compendium is available, reviewers may have difficulties reproducing the experiments due to missing libraries or dependences on a specific operating system version to run (or compile) the experiment. We posit that by planning for reproducibility and through the use of systems that systematically capture provenance of the scientific exploration process, researchers will not only create results that are reproducible but they can also streamline many of the tasks they have to carry out. With this in mind, we have built a framework that supports the life cycle of computational experiments [25, 38]. This framework has been implemented and is currently released as part of VisTrails [20, 70], an open-source, workflow-based data exploration and visualization system. VisTrails relies on a provenance management component to automatically and transparently capture the necessary metadata to allow experiments to be reproduced, including executable specification of computational processes (i.e., the workflow structure), parameter settings, input and output data, library versions, and code. It implements mechanisms that leverage the provenance information to support the exploratory process [37, 39, 60] which is common in data-intensive science [30]. These mechanisms also make it easier for reviewers to run and verify the results.

In this chapter, we describe the VisTrails reproducibility infrastructure. We start in Section 1.2 with a definition for computational reproducibility

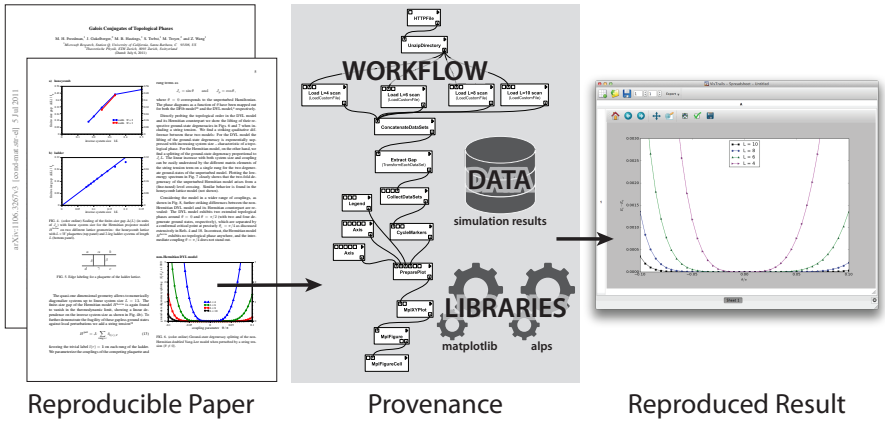

FIGURE 1.1
A reproducible paper. This paper by Freedman et al. [19] contains figures that have been inserted via the VisTrails publishing mechanisms. Clicking on a figure downloads the workflow instance and associated provenance needed to derive the figure. This information can be examined and executed in VisTrails, reproducing the plot shown in the figure.

and reproducibility levels. We also give an overview of workflow systems and discuss their benefits and limitations for the creation of reproducible experiments. The VisTrails system is described in Section 1.3, and in Section 1.4, we present specific components we have added to the system to support both authors and reviewers of reproducible experiments. Some limitations of our reproducibility framework and directions for future work are discussed in Section 1.5. We review related work in Section 1.6, and conclude in Section 1.7.

1.2 Reproducibility, Workflows, and Provenance

1.2.1 The Anatomy of a Reproducible Experiment

A computational experiment that has been developed at time t on hardware/operating system s on data d is reproducible if it can be executed at time t' on system s' on data d' that is similar to (or potentially the same as) d with consistent results [22]. For this to be possible, the description of the experiment must be sufficiently precise and include:

- A description of the input data, either in extension (the actual data) or in intention (e.g., a script that derives the data);
- Detailed information about the system where the experiments were run, including hardware and software configuration;

- An executable specification for the experiment that describes the steps followed to derive the result.

The components of a reproducible paper are illustrated in Figure 1.1. This paper investigates Galois conjugates of quantum double models [19]. Each figure in the paper is accompanied by its provenance, consisting of the workflow used to derive the plot, the underlying libraries invoked by the workflow, and links to the input data, i.e., simulation results stored in an archival site. This provenance information allows all results in the paper to be reproduced. In the PDF version of the paper,[1] the figures are active, and when clicked on, the corresponding workflow is loaded into the VisTrails and executed on the reader's machine. The reader may then modify the workflow, change parameter values, and input data.

Levels of Reproducibility. While full reproducibility is desirable, it can be hard or impossible to attain. Therefore, it is important to consider different levels of reproducibility. Freire et al. [22] have introduced three criteria to characterize the level of reproducibility of experiments:

1. The *depth* evinces how much of an experiment is made available. The default today is to include a set of figures in a manuscript. Higher depths can be obtained by including: the script (or spreadsheet file) used to generate the figures in the paper together with the appropriate data sets; the raw data and intermediate results derived during the experiments; the set of experiments (system configuration and initialization, scripts, workload, measurement protocol) used to produce the raw data; the software system as a white box (source, configuration files, build environment) or black box (executable) on which the experiments are performed.
2. The level of *portability* indicates whether the experiments can be reproduced (a) on the original environment (basically the author of the experiment can replay it on his or her machine); (b) on a similar environment (e.g., same OS but different machines), or (c) on a different environment (i.e., on a different OS or machine).
3. *Coverage* specifies how much of an experiment can be reproduced. Coverage can be partial or full. For example, considering an experiment that requires special hardware to derive data, partial reproducibility can be obtained by providing the data produced by the hardware and the analysis processes used to derive the plots included in the paper.

[1] This paper can be downloaded from http://arxiv.org/abs/1106.3267.

1.2.2 Describing Computations as Workflows

Workflows are widely used to represent and execute computational experiments, as evidenced by the emergence of several workflow-based systems, such as Apple's Mac OS X Automator, Yahoo! Pipes, Galaxy, NiPype, VisTrails, Kepler and Taverna, to name a few. Workflow systems have features that make them suitable as tools to create reproducible experiments. Notably: (i) workflow specifications provide an explicit representation of the structure of the experiments, (ii) workflows automate repetitive tasks and computations, and (iii) workflow systems can transparently capture provenance information.

In a workflow, computational steps are represented by modules and there is a connection between two modules if there is a dependency relation between them. The dependency relation can be either control or data driven. When the dependencies are data driven, workflows are referred to as *dataflows*. Dataflows can be naturally represented as directed-acyclic graphs (DAGs) where the connections (or edges) correspond to data flowing between modules. Dataflows are the underlying model for the major scientific workflow systems [35, 62, 70] and also for many workflow-based systems used for data processing and visualization.

Dataflows have several advantages over scripts or programs written in high-level languages [25]. They provide a simple programming model where a sequence of tasks is composed by connecting the outputs of one task to the inputs of another. This enables the use of visual interfaces that are suitable for users without programming expertise. The explicit DAG structure supports useful manipulations, including the ability to query workflows and update them in a programmatic fashion [57]. Workflows can also be represented at different levels of *abstraction* [23]. As illustrated in Figure 1.2, a series of modules can be grouped to hide unnecessary details. Abstraction can be used to make the specification easier to understand and more amenable for publication.

A given workflow instance embodies not only the structure of the experiment, but also its configuration, i.e., the input data and parameters used to produce a result. Having a workflow instance associated with a published result simplifies reproducibility. In addition, because the workflow specification is executable and has an explicit structure, users can easily perform parameter sweeps and run experiments varying the input data, while ensuring the same configuration is used across the different runs. For example, the VisTrails system provides a mechanism for parameter exploration and allows users to compare the results side by side [24]. Figure 1.3 shows a series of scatter plots showing temperature and precipitation derived for multiple years: the same workflow is run varying the input files for different years. Such a mechanism is useful, for instance, to verify results and perform sensitivity analyses, tasks that are essential for reviewers.

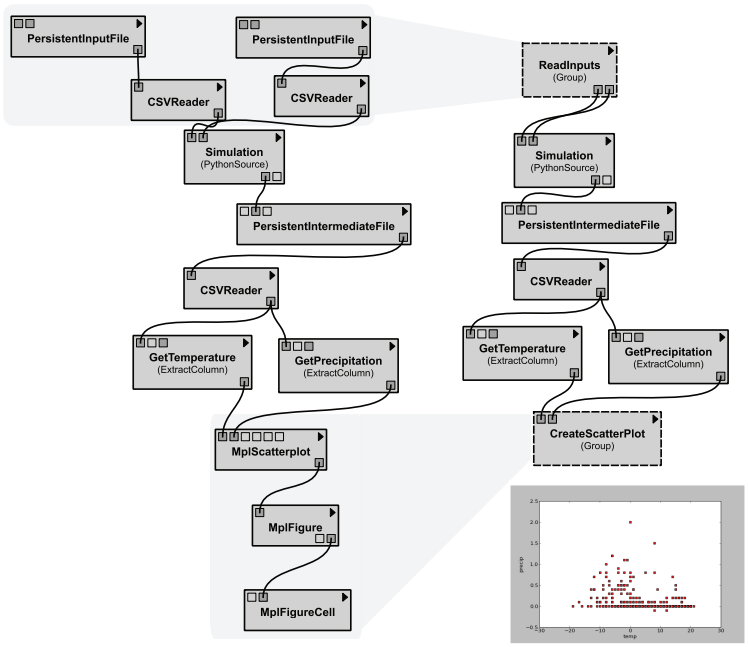

FIGURE 1.2
A scientific workflow for visualizing temperature and precipitation data. Besides providing an executable specification for the derivation of the scatterplot (bottom right), through the use of abstraction, the workflow can also provide a specification that hides unnecessary details and is easier to understand.

1.2.3 Provenance in Workflow Systems

In a script or a program, unless specified by the programmer, it is difficult to record the steps taken, the inputs consumed and the tools called throughout the execution. Because workflow systems control the execution of computational processes, they can systematically *capture their provenance*. As provenance is a key ingredient for reproducibility [23, 25, 60], workflows systems are well-suited as a platform to create reproducible experiments.

There are three main types of provenance captured by workflow systems: prospective, retrospective and workflow evolution [23]. *Prospective provenance* embodies the description of an experiment—the specification of the workflow structure, including modules, connections and inputs. *Retrospective provenance* captures information about the execution of the workflow, what actually happened when the workflow was run. *Workflow evolution* captures the history of a workflow, i.e., all the different versions of the workflow. Evolution provenance is especially useful for for data-intensive tasks, where workflows are iteratively refined, e.g., to experiment with different algorithms or simu-

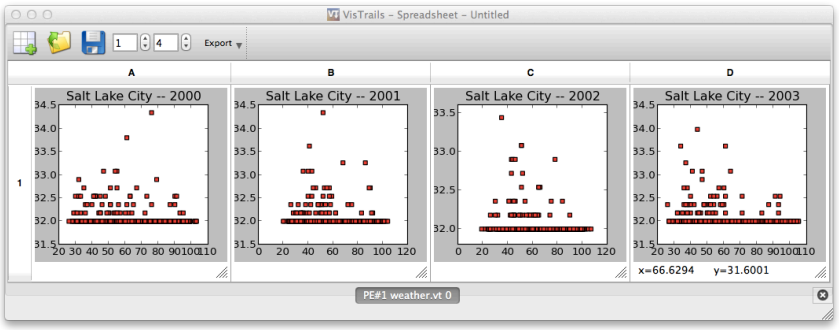

FIGURE 1.3
The VisTrails parameter exploration mechanism. A workflow is used to derive scatterplots for temperature and precipitation varying the input data to analyze the behavior in different years.

lation codes, different input data, etc. Interfaces can be created to allow users to navigate over workflow versions in an intuitive way, undo changes without losing results, visually compare multiple workflows and show their results side-by-side (see Figure 1.4).

1.2.4 Workflows and Reproducibility

Workflow systems capture both prospective and retrospective provenance, as such, they can provide a high depth of reproducibility: a detailed account of how a result was derived. Because the prospective provenance, i.e., the workflow specification, is executable, the experiments can be reproduced. One caveat is the fact that workflows may not be portable: it may not be possible to run a workflow in an environment different from the one where it was created. This can be due to a number of factors, including hard-coded file names, missing libraries, and OS incompatibility. In Section 1.4, we discuss how we have extended the VisTrails system to deal with these limitations.

1.3 The VisTrails System

VisTrails (`http://www.vistrails.org`) is an open-source provenance management and scientific workflow system that was designed to support the scientific discovery process [20, 24, 25]. VisTrails provides unique support for data analysis and visualization, a comprehensive provenance infrastructure, and a user-centered design. The system combines and substantially extends useful features of visualization and scientific workflow systems. Similar to vi-

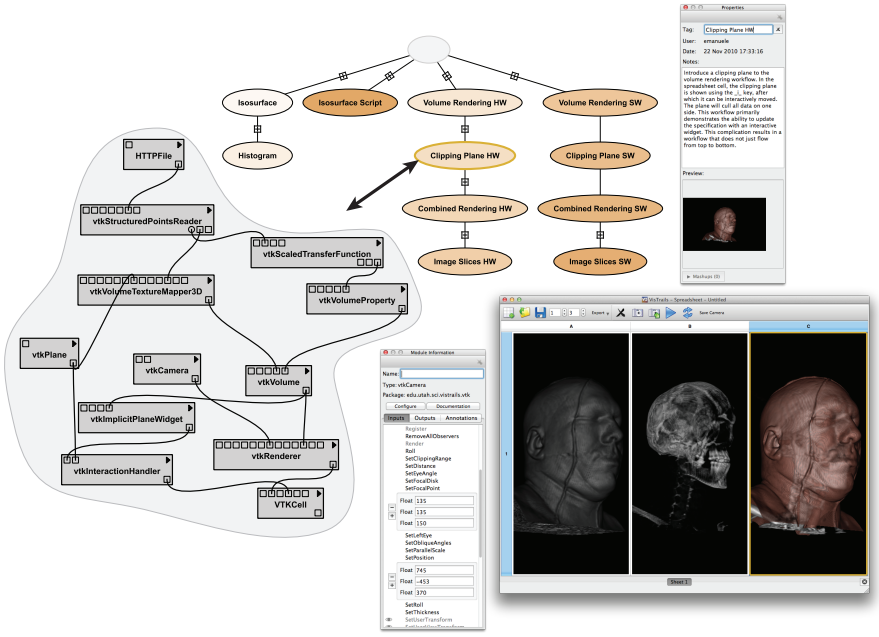

FIGURE 1.4
The VisTrails system consists of layers to manage computations, provenance, and data. The interface includes a version tree that allows users to navigate over past versions, a workflow view where users can create or modify workflows, and a visual spreadsheet where results can be compared side by side. Users may also add tags and annotations to the different workflow versions.

sualization systems [33, 36, 41, 66], VisTrails makes advanced visualization techniques available to users, allowing them to explore and compare different visual representations of their data; and similar to scientific workflow systems [35, 50, 62, 69], VisTrails enables the composition of workflows that combine specialized libraries, distributed computing infrastructure, and Web services. As a result, users can create complex workflows that encompass important steps of scientific discovery, from data gathering and manipulation, to complex analyses and visualizations, all integrated in one system.

Whereas workflows have been traditionally used to automate repetitive tasks, for applications that are exploratory in nature, such as simulations, data analysis and visualization, very little is repeated—change is the norm. As a scientist generates and evaluates hypotheses about data under study, a series of different, albeit related, workflows are created as they are adjusted in an iterative process. VisTrails was designed to manage these rapidly-evolving workflows. Another distinguishing feature of VisTrails is a *comprehensive provenance infrastructure* that maintains detailed history information about the steps followed and data derived in the course of an exploratory task [24]: Vis-

Trails maintains provenance of data products (e.g., visualizations, plots), of the workflows that derive these products, and their executions. The system also provides extensive annotation capabilities that allow users to enrich the automatically captured provenance. This information is persisted as XML files or in a relational database. Besides enabling reproducible results, VisTrails *leverages provenance information through a series of operations and intuitive user interfaces that aid users to collaboratively analyze data*. Notably, the system supports reflective reasoning by storing temporary results, by providing users the ability to reason about these results and to follow chains of reasoning backward and forward [47]. Users can navigate workflow versions in an intuitive way, undo changes but not lose any results, visually compare multiple workflows and show their results side-by-side in a visual spreadsheet, and examine the actions that led to a result [6, 24, 60]. In addition, the system has native support for parameter sweeps, whose results can also be displayed on the spreadsheet [24].

VisTrails addresses important usability issues that have hampered a wider adoption of workflow and visualization system. It provides a series of operations and user interfaces that simplify workflow design and use, including the ability to create and refine workflows by analogy, to query workflows by example, and a recommendation system that automatically suggests workflow completions as users interactively construct their workflows [40, 57]. The system also supports the creation of mashups—customized and simplified applications that can be more easily deployed to scientists [55, 56].

A beta version of the VisTrails system was first released in January 2007. Since then, the core system has been downloaded over 40,000 times. The VisTrails wiki has had over 1.7 million page views, and Google Analytics reports that visitors to the site come from 70 different countries. VisTrails has been adopted in several scientific projects, both nationally and internationally, and in different areas, including environmental science [4, 12, 31, 32], psychiatry [3], astronomy [63], cosmology [2], high-energy physics [16], molecular modeling [29], quantum physics [5, 19], earth observation [14, 68] and habitat modeling [46]. Besides being a stand-alone system, VisTrails has been used as a key component of domain-specific tools. One notable example is UV-CDAT, a new toolset for large-scale climate data analysis [54, 67].

A number of groups have contributed to the project, some directly—by checking in code into our git repository, and others by sharing packages that add functionality to VisTrails, for example: ALPS (ETH Zurich) [1], control flow [11] (Federal University of Rio de Janeiro, Brazil), ITK [34] (University of Utah), GridFields [28] (University of Washington), vtDV3D [72] (NASA), SAHM [46, 53] (USGS). Researchers at the Council for Scientific and Industrial Research (CSIR) in South Africa have added spatial-temporal data access and data pre-processing capabilities to VisTrails [18]. The system has been used by many projects, including DataONE [15] and STC-CMOP [12]. VisTrails has also been successfully used as a tool for teaching, having been adopted at universities in the United States and abroad [59].

1.4 Reproducing and Publishing Results with VisTrails

Because it was designed as a provenance-aware system, VisTrails natively stores much of the information necessary for users to revisit or extend existing work. To provide better support for result publication, we have added new functionality to VisTrails including support for portability (i.e., the ability to run a workflow in an environment different from the one where it was created) and to connect published results to their provenance [25, 38]. The new functionality both integrates with and complements the core provenance features. At the same time, they are not required for users of VisTrails nor are the ideas strictly dependent on VisTrails—they could be implemented on other systems.

1.4.1 Reproducibility Support

VisTrails captures both the provenance of workflow executions and the provenance of the workflow specifications. The system captures what happens when a workflow is run—the dependencies and properties of intermediate data, and how workflows are modified from run to run—which parameters are changed, which modules are connected together. Workflow evolution is a key element in maintaining reproducible results, capturing the changes that are made from initial explorations, tests and extensions to the final published results. Because no workflow versions are deleted or replaced, all results can be retrieved, reproduced, and compared against. At the same time, knowing how a computation was built can help others understand the process, extend the results or tweak them in meaningful ways.

Provenance-Rich Results Not only does VisTrails capture provenance, but it also makes this information available to users to help them organize and understand past work. This can be especially useful in collaborative settings, where multiple users are contributing ideas and making changes. In addition to a visual workflow builder, VisTrails provides a version tree that displays all of the workflow versions users have created and their relationships. A version tree is shown in Figure 1.4. Each node in the tree corresponds to a workflow version; an edge corresponds to an action or sequence of actions applied to transform the parent node into the child. For example, the node tagged `Clipping Plane HW` was created by modifying the node `Volume Rendering HW` to include a clipping plane. Unlike standard undo stacks, VisTrails captures and maintains all of the steps a user has taken, regardless of whether they may have been unproductive. A user can switch to any version by selecting the appropriate node in the version tree. Having access to this workflow evolution information allows users to investigate any idea they wish without worrying about explicitly saving versions along the way. In addition, as illustrated in Figure 1.4 (top right), each version is associated to metadata that includes

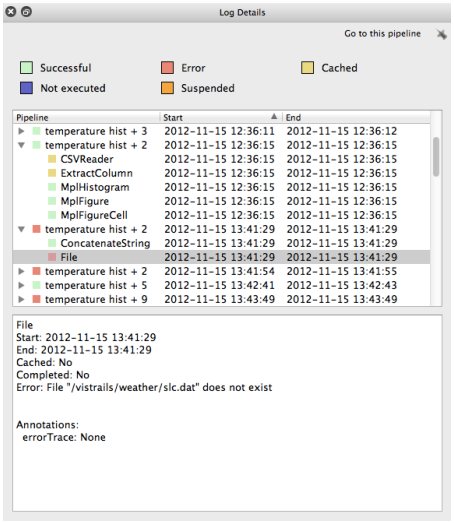

FIGURE 1.5
Retrospective provenance in VisTrails. Detailed information is kept about the execution of workflows and their modules, including when and for how long they ran, and whether their execution completed or failed.

an optional label (tag) that describes the version, information about the user who created the version, the time/date of creation, and free-text notes that users can add to provide further details about their findings and to better document the exploratory process. Because both the workflow specification and metadata can be searched (and queried), users also do not have to worry about meticulously labeling each version with the exact parameters and inputs used. This reduces the burden on users to maintain all of the information needed for reproducibility.

In addition to workflow evolution provenance, VisTrails also allows users to view the provenance of workflow executions (see Figure 1.5). This provenance information, containing timing information, any errors encountered, and which system was used, can be valuable in diagnosing possible issues or determining more efficient methods of execution. It can also be used to determine which outputs used a particular input, making it possible to highlight results that may be invalidated by, for example, a malfunctioning sensor. Such information is also important for reproducibility because it is possible that the system being used or any previous errors can inform those who attempt to emulate the original work later. For this reason, a vistrail contains a set of related workflows, the changes differentiating one from another, and the provenance of any executions.

Workflow Upgrades As time progresses, workflows can become stale for a number of reasons, notably due to the fact that the software the workflow

relies upon may change. This can happen both during the scientific exploration or after results are published. When possible, we wish to retain the original steps so that given a compatible system (e.g., a virtual machine), they can be repeated exactly. However, for later use, it is more convenient to be able to work with the original computations in a current system. This requires both a recognition of outdated computations and an upgrade path.

There are a variety of changes that may have occurred to warrant an upgrade. It may be a change in algorithm where the interface (parameters, input and output ports) remains the same. It could also be that the ports were changed to provide new parameters or outputs, possibly with relabeling. Finally, a module (or network of modules) may have been replaced or removed due to a reorganization or reimplementation. All of these types of upgrades can be handled, but some can be defaulted to automatic steps while others require developer or user involvement.

VisTrails detects when upgrades are necessary by comparing the version of a module in a given workflow with the currently available version. If the module is out-of-date, it tries to upgrade the old module. Upgrade paths are either determined automatically or specified by the module developer [39]. When the interface has not changed, VisTrails replaces the old module with the new version. When it has, VisTrails will attempt to replace the old module, and when ports have only been added or a changed port has not been used, it will succeed. When it does not, it alerts the user so he/she can determine a next step. However, for non-trivial upgrades, developers are encouraged to provide explicit upgrade paths for the modules in their packages. VisTrails passes along any modules that the developer has designated for special handling, and the developer can write a set of changes (the normal VisTrails actions like "add module", "change parameter", etc.) to be applied. These changes can be based on the current parameters being sent to the module as well as any neighboring modules. When automated upgrades do not work and developers have not specified an upgrade path, VisTrails alerts the user who can then make the necessary changes.

Most importantly, any upgrades are recorded with the same change-based provenance as a normal action. This means the original version of the workflow is always retained, and anyone looking at the steps in the future can see exactly how the original workflow was modified. One could, then, re-execute the original version in a virtual machine and compare it to an upgraded version to check if the behavior is unchanged or if a specific bug has been fixed.

Managing Data and their Versions Even if authors have maintained the specification of the computation needed to reproduce their results, reproducibility also requires the data used in the work. The provenance of a computation may indicate the filename used, but if this file is moved, deleted, or modified, reproducing that work is not possible. One value we have added to VisTrails provenance is a hash of a file's contents, allowing users to later check if the provided data does indeed match the one used in the original computa-

tion. In addition, we have developed a *persistence package* that allows users to store input, output, and intermediate data in a versioned repository [37]. This repository not only ensures that data can be accessed later, but it also automatically tracks changes in data, ensuring that different versions of the input data can be recovered in the future. If a user generated a figure based on a data set which was later updated, the user can go back to the original version and reproduce the original run. This mechanism also creates strong links between output data and the computations used to generate them: users can match output data and computations by comparing hashes with the repository and finding provenance traces that match the given identifiers.

Scientific workflows may also access external data sources such as relational databases as part of an experiment. For example, a module may have to remotely query a large dataset and use the results for the remaining computation, or it may also have to update information in the dataset. As with file-based references, accessing relational databases in workflows may also lead to problems for reproducibility. For example, when workflow consumes data from a database, its results may depend on the data. When the database is updated, the results of the workflow may no longer be reproduced, as database updates may have changed the data that is used by the workflow. Reproducibility, in this case, is more challenging because there is an inherent mismatch between workflow and database models: while workflows are stateless and deterministic, databases are stateful—new states reflect the changes applied to the database.

To address this issue, we have implemented in VisTrails a model that integrates workflow and database provenance and enables reproducibility for workflows that interact with relational databases [9]. We rely on a transaction temporal model that is currently supported by commercial RDMS—in our implementation, we used Oracle [48] and its Total Recall functionality [49]. The states of the database are systematically captured and added to the workflow provenance, and this information is used by VisTrails to communicate with the database and go back to a previous state so the workflow can be correctly reproduced.

Using Provenance for Future Work Provenance allows users to go back to previous work and possibly extend it, but it can also be used to help create or inform future work. For example, if a user performs some modification to a workflow like adding a data filtering step, she may wish to add a similar step to other workflows as well. This process could be very tedious, but because the VisTrails provenance contains the steps needed to transform the workflow, it can usually be automated. VisTrails workflow analogies [57] allow users to modify a set of workflows based on the changes they have made to a single workflow; the technique uses a flexible matching algorithm that allows the changes to be applied to workflows that have different structures. While analogies are useful for users that have well-defined changes, provenance can also be mined to derive common workflow patterns. Similar to completion

techniques often seen in text entry boxes, VisComplete [40] uses a collection of workflows to build ranked sets of possible completions for a partially constructed workflow. This allows users to save time spent rebuilding common substructures or searching examples. These techniques highlight the use of provenance not only as a record of past work, but also as the starting point for future exploration.

1.4.2 Publishing Results

One common problem with published work is that the caption of a figure, table, or other result does not provide the whole story about the origin of that figure. For example, after plotting raw data, an author may restrict the dataset, transform the data to emphasize a particular aspect, or just refine the graphical presentation. Sometimes, these steps are not recorded and later, readers (or even the original authors) are hard-pressed to determine all the steps used to generate the result. Our approach is to encode the actual computation as part of the paper so that upon generating the final PDF, the result is recalculated if necessary and a link to the exact computation included.

Our implementation of this hard link between the results displayed in a paper and the computation uses VisTrails, LaTeX, and a LaTeX package that defines a new command to reference the underlying computation and passes the information to VisTrails for computation, placing the result from VisTrails directly into the paper [38]. The `vistrails` LaTeX package defines the base command for inserting a result as well as options for adding links to a Web-hosted definition of the workflow or an interactive version of the result. The user can specify the workflow used to generate a result using a tag or a unique identifier. With tags, the computation can be edited and the paper, upon recompilation, will include the updated result automatically. We have also implemented mechanisms to include results derived by VisTrails into Web pages, wikis, Word documents and PowerPoint presentations. Having these mechanisms reduces the burden on authors to manually update results and mitigates the problem of losing previous results.

1.4.3 Publishing Interactive Results on the Web

The Web has opened the possibility of publishing much more than a printable PDF, but the same issues that arise in traditional publishing must be addressed. Specifically, authors must make sure that the published result accurately reflects the underlying computation, and the idea of reproducibility as inspection and further exploration is not met simply by having an interactive visualization. As with papers, being able to download, execute, and modify a result is preferred over a static result locked into a particular site. That said, there are benefits of server-hosted results, and many of the goals of reproducibility can be met in such an environment. Because users do not need to install additional software or download large datasets, the burden of explor-

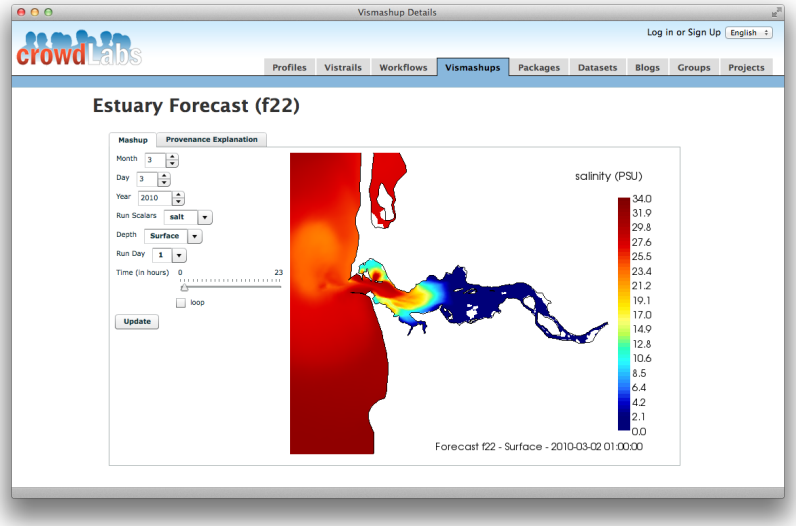

FIGURE 1.6
Authors may publish interactive results on the Web. Here, we show how VisTrails results can be displayed and interacted with on the crowdLabs site.

ing results is lessened. Furthermore, the interactive possibilities on the Web allow authors to publish results that permit recomputation with user-defined inputs and parameters.

With VisTrails, we have explored two alternatives for Web-based publishing: wikis and interactive social Web sites. Our approach for wikis is very similar to publishing for traditional PDFs: there is a specific `vistrails` tag that allows a user to specify the workflow that should be computed to generate the output. As with LaTeX, a user can indicate the workflow by a tag or a specific, unique identifier. After editing a wiki page, there is a MediaWiki extension that processes the `vistrail` tag to re-execute, if necessary, the workflow and insert the result directly into the Wiki output.

We have also developed the crowdLabs Web site as a place where users can host and share their workflows and results [45]. In addition, crowdLabs supports VisTrails mashups which allow workflow creators to easily specify higher-level interfaces to workflows [56]. Such interfaces make it easy for users to quickly explore different parameter settings for a given computation. Users can upload an entire version tree (with all its tagged workflows), a specific workflow, datasets, or a mashup directly from the VisTrails application. The computations can then be executed on the crowdLabs server and the results made available via a Web browser. For mashups, a user can modify inputs to generate new results on the fly without having VisTrails installed locally. This

server-side execution allows some amount of reproducibility without requiring a reader to download the required packages and data. Furthermore, crowdLabs allows users to comment on others' results and approaches, enabling conversations about the computations that can help further future extensions.

1.5 Challenges and Opportunities

In the previous section, we have described our efforts in the area of computational reproducibility. The infrastructure we have presented, along with all its features, has already been successfully used by different research groups [38]. However, the infrastructure is by no means comprehensive. Through our collaborations with scientists, we have gathered a set of requirements that guided our design. In the long-term, our goal is to build a general system where different components and methods can be mixed and used to achieve reproducibility for many different domains and scenarios.

Our infrastructure is built on top of the VisTrails system, but this tight integration is not always desirable. Although there are significant advantages for using workflows (Section 1.2.2), it may be time-consuming to wrap the experiment into a workflow system if scientists are already using another approach for the execution. Besides, the experiment may involve interactive tools, which cannot be wrapped into a workflow. Nonetheless, the key contribution here are the core ideas and functionalities on which the infrastructure is based—the infrastructure itself is a proof-of-concept implementation of our efforts to simplify the creation, review and reuse of reproducible experiments. A direction we are currently pursuing is to break this infrastructure into components that can be more easily integrated with other systems. In addition, we have developed a plug-in mechanism that leverages the VisTrails provenance management subsystem to provenance support to other tools. Examples of such tools are VisIt, Autodesk's Maya and ParaView [8].

1.6 Related Work

There are many scientific workflow systems, besides VisTrails, that represent computations as dataflows, including Swift [61], Taverna [62], Kepler [35], Triana [64] and Pegasus/Wings [50]. While these systems support reproducibility, they do not have support for portability as discussed in Section 1.2.4.

There are also a number of other tools that provide support for reproducibility. Madagascar [44] is a software package for geophysics that allows scientists to generate computational results and include these on reproducible

documents by making use of SCons, a software construction tool, and LaTeX. Sweave [42] is a tool that embeds source code from R, the statistical computing software, in LaTeX documents. In this way, every time data and analysis change, the document is automatically updated, creating *dynamic reports*, which supports reproducible research. `ReproZip` is a tool that automatically captures the provenance of existing experiments and packs all the components necessary to reproduce the results in different environments [10]. Reviewers can then unpack and run the experiments in their environment without having to install any additional software. `ReproZip` also generates a workflow specification for the experiment which reviewers can use to explore the experiment and try different configurations.

The idea of linking data to publications is also explored by SOLE, Collage, SHARE and VCR. SOLE [51] is a system that defines command-line tools to create scientific objects, such as source code, annotations in PDFs and virtual machine images hosted on a cloud, and link those objects to a paper in HTML format. Collage [13] is a framework, integrated by the publisher Elsevier, that was developed to create executable papers, where authors include their code and data. SHARE [27] is a web portal that allows authors to create and share remote virtual machines. These machines can be cited in research papers, and readers can access them and fully reproduce the experiment. Last, Verifiable Computational Results, or VCRs [26], are computational results that have an identifier, known as Verifiable Result Identifier (VRI), which is a URL that points to a repository where the results and the computational process behind it are located. VCRs can then be published in papers, and reviewers and readers may follow the VRIs to possibly reproduce the results.

1.7 Conclusion

End-to-end and long-term reproducibility of a scientific result is hard to achieve due to the factors that include the use of specialized hardware, proprietary data, and inevitable changes in hardware and software environments. Nonetheless, with the infrastructure we have built, it is possible to accurately document the processes through provenance capture, as well as to attain reproducibility for important sub-components of a result, such as, for example, the analysis and visualization of data derived from simulations run on special hardware.

As reproducibility becomes more widely adopted, the availability of repositories that contain fully-documented experiments will open up new opportunities for scientific sharing and progress. These repositories have the potential to streamline scientific discovery by allowing researchers to search through and more easily re-use existing work [21].

Acknowledgments

We thank the VisTrails team for making this work possible, especially Emanuele Santos, Tommy Ellqvist and Huy T. Vo. We also thank our many collaborators and users that have provided us feedback on our reproducibility infrastructure, particularly Philippe Bonnet, Dennis Shasha, Joel Tohline and Matthias Troyer. The research and development of the VisTrails system has been funded by the National Science Foundation under grants CNS-1229185, IIS-1139832, IIS-1142013 IIS-0905385 IIS 1050422, IIS 0844572, ATM-0835821, IIS-0844546, IIS-0746500, CNS-0751152, IIS-0713637, OCE-0424602, IIS-0534628, CNS-0514485, IIS-0513692, CNS-0524096, CCF-0401498, OISE-0405402, CCF-0528201, CNS-0551724, the Department of Energy SciDAC (VACET and SDM centers), and IBM Faculty Awards.

About the Authors

Juliana Freire is a Professor at the Department of Computer Science and Engineering at the Polytechnic Institute of New York University. She also holds an appointment in the Courant Institute for Mathematical Science. Before, she was an Associate Professor at the School of Computing, University of Utah; an Assistant Professor at OGI/OHSU; and a member of technical staff at the Database Systems Research Department at Bell Laboratories (Lucent Technologies). Her recent research has focused on Web-scale data integration, big-data analysis and visualization, and provenance management. She has been actively involved in the development of tools and infrastructure to support reproducible research and she has also been part of the steering committee for the ACM SIGMOD Experimental Reproducibility Effort. She is a recipient of an NSF CAREER and an IBM Faculty award. Her research has been funded by grants from the National Science Foundation, Department of Energy, National Institutes of Health, the University of Utah, Sloan Foundation, Amazon, Microsoft Research, Yahoo! and IBM. Contact her at `juliana.freire@nyu.edu`.

David Koop is a Research Assistant Professor at the Department of Computer Science and Engineering at the Polytechnic Institute of New York University. In the past, he was a Senior Software Architect at VisTrails, Inc. where he worked on developing provenance solutions for new and existing software. He received his PhD in Computing from the University of Utah and completed a Masters at the University of Wisconsin-Madison. His research interests span the areas of visualization and scientific data management, and he has been focused on how provenance information can be used to improve data explo-

ration and analysis. Contact him at dkoop@poly.edu.

Fernando Chirigati is a Research Assistant and Graduate Student at the Department of Computer Science and Engineering at Polytechnic Institute of New York University. Before, he was a Research Assistant at the Federal University of Rio de Janeiro, Brazil. His research interests are mainly in the area of scientific data management, including provenance, scientific workflows, reproducibility and data visualization. He has a BE in Computer and Information Engineering from the Federal University of Rio de Janeiro, Brazil. Contact him at fchirigati@nyu.edu.

Cláudio T. Silva is Head of Disciplines of the NYU Center for Urban Science and Progress (CUSP), Professor of Computer Science and Engineering at NYU Poly, affiliate faculty at NYU Courant, and an adjunct professor at the University of Utah. Dr. Silva has co-authored more than 200 technical papers and 11 U.S. patents, primarily in visualization, geometry processing, computer graphics, scientific data management, HPC, and related areas. His current research interests include Big Data and urban systems, visualization and data analysis, and geometry processing. He has served on more than 100 program committees, and he is currently on the editorial board of the ACM Transactions on Spatial Algorithms and Systems (TSAS), Computer Graphics Forum, Computing in Science and Engineering, Computer and Graphics, The Visual Computer, and Graphical Models. Dr. Silva was general co-chair of IEEE VisWeek 2010, and papers co-chair of IEEE Visualization 2005 and 2006. He received IBM Faculty Awards in 2005, 2006, and 2007, and several best paper awards. He is a Fellow of the IEEE. His research has been funded by grants from the National Science Foundation, Department of Energy, National Institutes of Health, the University of Utah, Sloan Foundation, Sandia, LLNL, ExxonMobil and IBM. Contact him at csilva@nyu.edu.

Bibliography

[1] The ALPS project. `http://alps.comp-phys.org`.

[2] Erik W. Anderson, James P. Ahrens, Katrin Heitmann, Salman Habib, and Cláudio T. Silva. Provenance in comparative analysis: A study in cosmology. *Computing in Science and Engineering*, 10(3):30–37, 2008.

[3] Erik W. Anderson, Gil A. Preston, and Cláudio T. Silva. Towards development of a circuit based treatment for impaired memory: A multidisciplinary approach. In *IEEE EMBS Neural Engineering*, pages 302–305, 2007.

[4] António Baptista, Bill Howe, Juliana Freire, David Maier, and Cláudio T. Silva. Scientific exploration in the era of ocean observatories. *Computing in Science and Engineering*, 10(3):53–58, 2008.

[5] B. Bauer et al. The ALPS project release 2.0: open source software for strongly correlated systems. *Journal of Statistical Mechanics: Theory and Experiment*, 2011(05):P05001, 2011.

[6] Louis Bavoil, Steve Callahan, Patricia Crossno, Juliana Freire, Carlos Scheidegger, Cláudio Silva, and Huy Vo. VisTrails: Enabling interactive multiple-view visualizations. In *Proceedings of IEEE Visualization*, pages 135–142, 2005.

[7] Biostatistics journal. http://biostatistics.oxfordjournals.org.

[8] Steven P. Callahan, Juliana Freire, Carlos E. Scheidegger, Cláudio T. Silva, and Huy T. Vo. Towards Provenance-Enabling ParaView. In Juliana Freire, David Koop, and Luc Moreau, editors, *Provenance and Annotation of Data and Processes*, Lecture Notes in Computer Science, pages 120–127. Springer-Verlag, 2008.

[9] Fernando Chirigati and Juliana Freire. Towards integrating workflow and database provenance. In Paul Groth and James Frew, editors, *Provenance and Annotation of Data and Processes*, Lecture Notes in Computer Science, pages 11–23. Springer Berlin, 2012.

[10] Fernando Chirigati, Dennis Shasha, and Juliana Freire. Reprozip: Packing experiments for sharing and publication. In *Proceedings of the 2013 ACM SIGMOD International Conference on Management of Data*, SIGMOD '13, 2013. To appear.

[11] Fernando Seabra Chirigati, Rafael Dahis, Sergio Manuel Serra da Cruz, Juliana Freire, Cláudio Silva, and Marta Mattoso. Desenvolvimento de estruturas de controle explícito para o SGWfC VisTrails. In *Brazilian Symposium on Databases (SBBD)*, 2009. Best poster award.

[12] NSF Center for Coastal Margin Observation and Prediction (CMOP). `http://www.stccmop.org`.

[13] Collage: Authoring Environment for Executable Publications. `https://collage.elsevier.com/`.

[14] Council for Scientific and Industrial Research (CSIR) in South Africa. `http://www.csir.co.za`.

[15] The Data Observation Network for Earth (DataONE). `https://dataone.org/`.

[16] Andrew Dolgert, Lawrence Gibbons, Christopher D. Jones, Valentin Kuznetsov, Mirek Riedewald, Daniel Riley, Gregory J. Sharp, and Peter Wittich. Provenance in high-energy physics workflows. *Computing in Science and Engineering*, 10(3):22–29, 2008.

[17] D.L. Donoho, A. Maleki, I.U. Rahman, M. Shahram, and V. Stodden. Reproducible research in computational harmonic analysis. *Computing in Science & Engineering*, 11(1):8–18, Jan.-Feb. 2009.

[18] EO4VisTrails – Earth Observation Capabilities for VisTrails. `http://code.google.com/p/eo4vistrails`.

[19] M. H. Freedman, J. Gukelberger, M. B. Hastings, S. Trebst, M. Troyer, and Z. Wang. Galois conjugates of topological phases. *Phys. Rev. B*, 85:045414, Jan 2012.

[20] J. Freire, D. Koop, E. Santos, C. Scheidegger, C. Silva, and H. T. Vo. *The Architecture of Open Source Applications*, chapter VisTrails. Lulu.com, 2011.

[21] Juliana Freire, Philippe Bonnet, and Dennis Shasha. Exploring the coming repositories of reproducible experiments: Challenges and opportunities. *PVLDB*, 4(12):1494–1497, 2011.

[22] Juliana Freire, Philippe Bonnet, and Dennis Shasha. Computational reproducibility: state-of-the-art, challenges, and database research opportunities. In *Proceedings of the 2012 ACM SIGMOD International Conference on Management of Data*, SIGMOD '12, pages 593–596, New York, NY, USA, 2012. ACM.

[23] Juliana Freire, David Koop, Emanuele Santos, and Cláudio T. Silva. Provenance for computational tasks: A survey. *Computing in Science and Engg.*, 10(3):11–21, May 2008.

[24] Juliana Freire, Cláudio Silva, Steve Callahan, Emanuele Santos, Carlos Scheidegger, and Huy Vo. Managing rapidly-evolving scientific workflows. In *International Provenance and Annotation Workshop (IPAW)*, LNCS 4145, pages 10–18. Springer Verlag, 2006.

[25] Juliana Freire and Cláudio T. Silva. Making computations and publications reproducible with VisTrails. *Computing in Science and Engineering*, 14(4):18–25, 2012.

[26] Matan Gavish and David Donoho. A universal identifier for computational results. *Procedia Computer Science*, 4(0):637 – 647, 2011. Proceedings of the International Conference on Computational Science (ICCS).

[27] Pieter Van Gorp and Steffen Mazanek. SHARE: a web portal for creating and sharing executable research papers. *Procedia Computer Science*, 4(0):589 – 597, 2011. Proceedings of the International Conference on Computational Science (ICCS).

[28] GridFields. `http://code.google.com/p/gridfields`.

[29] Randy Heiland, Maciek Swat, Benjamin Zaitlen, James Glazier, and Andrew Lumsdale. Workflows for parameter studies of multi-cell modeling (HPC). In *Proceedings of the ACM High Performance Computing Symposium*, pages 94:1–94:6, 2010.

[30] Tony Hey, Stewart Tansley, and Kristin Tolle, editors. *The Fourth Paradigm: Data-Intensive Scientific Discovery*. Microsoft Research, 2009.

[31] Bill Howe, Peter Lawson, Renee Bellinger, Erik Anderson, Emanuele Santos, Juliana Freire, Carlos Scheidegger, António Baptista, and Cláudio Silva. End-to-end escience: Integrating workflow, query, visualization, and provenance at an ocean observatory. In *IEEE International Conference on eScience*, pages 127–134, 2008.

[32] Bill Howe, Cláudio Silva, and Juliana Freire. A science cloud on your desktop: VisTrails + GridFields, 2009. `http://clue.cs.washington.edu`.

[33] IBM. OpenDX. `http://www.research.ibm.com/dx`.

[34] The insight toolkit. `http://www.itk.org`.

[35] The Kepler Project. `http://kepler-project.org`.

[36] Kitware. ParaView. `http://www.paraview.org`.

[37] David Koop, Emanuele Santos, Bela Bauer, Matthias Troyer, Juliana Freire, and Cláudio T. Silva. Bridging workflow and data provenance using strong links. In *SSDBM*, pages 397–415, 2010.

[38] David Koop, Emanuele Santos, Phillip Mates, Huy T. Vo, Philippe Bonnet, Bela Bauer, Brigitte Surer, Matthias Troyer, Dean N. Williams, Joel E. Tohline, Juliana Freire, and Cláudio T. Silva. A provenance-based infrastructure to support the life cycle of executable papers. *Procedia Computer Science*, 4:648–657, 2011. Proceedings of the International Conference on Computational Science (ICCS).

[39] David Koop, Carlos Scheidegger, Juliana Freire, and Cláudio T. Silva. The provenance of workflow upgrades. In *IPAW*, pages 2–16, 2010.

[40] David Koop, Carlos Eduardo Scheidegger, Steven P. Callahan, Juliana Freire, and Cláudio T. Silva. VisComplete: Automating suggestions for visualization pipelines. *IEEE Transactions on Visualization and Computer Graphics*, 14(6):1691–1698, 2008.

[41] Lawrence Livermore National Laboratory. VisIt: Visualize It in Parallel Visualization Application. `https://wci.llnl.gov/codes/visit` [29 March 2008].

[42] F. Leisch. Sweave: Dynamic generation of statistical reports using literate data analysis. In *Compstat*, pages 575–580, 2002.

[43] R.J. LeVeque. Python tools for reproducible research on hyperbolic problems. *Computing in Science & Engineering*, 11(1):19–27, Jan.-Feb. 2009.

[44] Madagascar. `http://www.ahay.org/wiki/Main_Page`.

[45] Phillip Mates, Emanuele Santos, Juliana Freire, and Cláudio T. Silva. CrowdLabs: Social analysis and visualization for the sciences. In *SSDBM*, pages 555–564, 2011.

[46] J. Morisette, C. Jarnevich, T. Holcombe, C. Talbert, D. Ignizio, M. Talbert, C. T. Silva, D. Koop, A. Swanson, and N. Young. VisTrails SAHM: Visualization and workflow management for ecological niche modeling. *Ecography*, 2013. To appear.

[47] Donald A. Norman. *Things That Make Us Smart: Defending Human Attributes in the Age of the Machine*. Addison Wesley, 1994.

[48] Oracle Database 11g Release 2. `http://www.oracle.com/technetwork/database/enterprise-edition/overview`.

[49] Oracle Total Recall with Oracle Database 11g Release 2. `http://www.oracle.com/technetwork/database/application-development/total-recall-1667156.html`.

[50] The Pegasus Project. `http://pegasus.isi.edu/`.

[51] Quan Pham, Tanu Malik, Ian Foster, Roberto Di Lauro, and Raffaele Montella. SOLE: Linking Research Papers with Science Objects. In Paul Groth and James Frew, editors, *Provenance and Annotation of Data and Processes*, volume 7525 of *Lecture Notes in Computer Science*, pages 203–208. Springer Berlin, 2012.

[52] PNAS Submission Guidelines. `http://www.pnas.org/site/misc/iforc.shtml\#submission`.

[53] Software for Assisted Habitat Modeling Package for VisTrails (SAHM: VisTrails). `http://www.fort.usgs.gov/products/software/sahm`.

[54] Emanuele Santos, David Koop, Thomas Maxwell, Charles Doutriaux, Tommy Ellqvist, Gerald Potter, Juliana Freire, Dean Williams, and CludioT. Silva. Designing a provenance-based climate data analysis application. In Paul Groth and James Frew, editors, *Provenance and Annotation of Data and Processes*, volume 7525 of *Lecture Notes in Computer Science*, pages 214–219. Springer Berlin, 2012.

[55] Emanuele Santos, David Koop, Huy T. Vo, Erik W. Anderson, Juliana Freire, and Cláudio T. Silva. Using workflow medleys to streamline exploratory tasks. In *SSDBM*, pages 292–301, 2009.

[56] Emanuele Santos, Lauro Lins, James Ahrens, Juliana Freire, and Cláudio T. Silva. VisMashup: Streamlining the creation of custom visualization applications. *IEEE Transactions on Visualization and Computer Graphics*, 15(6):1539–1546, 2009.

[57] Carlos Eduardo Scheidegger, Huy T. Vo, David Koop, Juliana Freire, and Cláudio T. Silva. Querying and creating visualizations by analogy. *IEEE Transactions on Visualization and Computer Graphics*, 13(6):1560–1567, 2007.

[58] SIGMOD Experimental Repeatability. `http://www.sigmod2011.org/calls_papers_sigmod_research_repeatability.shtml`.

[59] Cláudio T. Silva, Erik Anderson, Emanuele Santos, and Juliana Freire. Using VisTrails and provenance for teaching scientific visualization. *Computer Graphics Forum*, 30(1):75–84, 2011.

[60] Cláudio T. Silva, Juliana Freire, and Steven P. Callahan. Provenance for visualizations: Reproducibility and beyond. *Computing in Science and Engg.*, 9(5):82–89, September 2007.

[61] The Swift System. `http://www.ci.uchicago.edu/swift`.

[62] The Taverna Project. `http://www.taverna.org.uk/`.

[63] Joel E. Tohline, Jinghya Ge, Wesley Even, and Erik Anderson. A customized python module for CFD flow analysis within VisTrails. *Computing in Science and Engineering*, 11(3):68–73, 2009.

[64] The Triana Project. `http://www.trianacode.org`.

[65] IEEE Transactions on Signal Processing - Reproducible Research. `http://www.signalprocessingsociety.org/publications/periodicals/tsp/`.

[66] Craig Upson et al. The application visualization system: A computational environment for scientific visualization. *IEEE Computer Graphics and Applications*, 9(4):30–42, 1989.

[67] Ultrascale Visualization - Climate Data Analysis Tools (UV-CDAT). `http://uv-cdat.llnl.gov`.

[68] TL Van Zyl, G McFerren, and A Vahed. Earth observation scientific workflows in a distributed computing environment. Technical Report 7727, CSIR, 2011. http://hdl.handle.net/10204/5435.

[69] VDS - The GriPhyN Virtual Data System. `http://www.ci.uchicago.edu/wiki/bin/view/VDS/VDSWeb/WebMain`.

[70] VisTrails. http://www.vistrails.org.

[71] VLDB Experimental Reproducibility. `http://www.vldb.org/2013/experimental_reproducibility.html`.

[72] vtDV3D VisTrails Package. `http://portal.nccs.nasa.gov/DV3D/vtDV3D/_build/html/index.html`.